

LeanSplat: Trading Per-Frame Fidelity for Real-Time Gaussian-based SLAM

Abstract: Real-time mapping is core to robotic navigation and scene understanding. Gaussian splatting is a compelling SLAM representation: it pairs explicit geometry with differentiable rasterization. However, under tight latency constraints, the per-pixel compositing becomes a bottleneck: each pixel composites over many depth-sorted gaussians. The cost lies not in the arithmetic of compositing but the number of times SLAM has to invoke the renderer, the number of pixels rendered and gaussians per pixel.

We propose *LeanSplat*, a SLAM-oriented splatting approach that trades per-frame photorealism for real-time mapping. *LeanSplat* evaluates two approaches: image downscaling which cuts spatial workload by rendering fewer pixels, and early transmittance termination that halts compositing once gaussian contributions become negligible. We show that *LeanSplat* preserves dominant scene structure while eliminating latency-critical work. Integrated into a SLAM pipeline, *LeanSplat* speeds up mapping by up to $7.35\times$ while maintaining 0.74cm ATE at 50 frames. We do find that *LeanSplat* should be applied asymmetrically: conservatively for tracking and more aggressively for mapping.

1 Introduction

Simultaneous Localization and Mapping (SLAM) must jointly estimate sensor pose and build a map of the environment in real time, a basic requirement for any autonomous system [1]. The map representation is a central choice: it shapes tracking, mapping, and every downstream task such as planning and reconstruction. The representations have evolved from sparse landmarks [2, 3, 4] to dense geometry [5, 6]. This has raised computational cost; in particular, the volumetric ray marching that underlies NeRF-based SLAM makes real-time operation difficult [7, 8, 9, 10, 11]. 3D gaussian splatting [12] has emerged as an explicit volumetric representation that exploits fast rasterization, offering a path toward real-time SLAM.

Gaussian splatting represents a scene as anisotropic gaussians with explicit positions, opacities, and colors, an explicit and differentiable representation. Rendering is rasterization-based: gaussians are projected, depth-sorted per tile, and composited front-to-back. Recent SLAM systems [13, 14, 15] use it for joint tracking and mapping. The bottleneck, however, is compositing: each pixel accumulates contributions from many depth-sorted gaussians, and this rendering repeats across every optimization iteration. On Replica room0, the median pixel processes roughly 77 gaussian contributors and the 90th percentile roughly 107 before transmittance becomes negligible (Figure 2). As scenes grow, per-frame cost pushes iteration times well beyond real-time budgets on the embedded hardware used in robotics, such as the NVIDIA Thor platform. A natural response is to accelerate compositing itself (see § 4.2). However, we find they yield only marginal speedup while destabilizing optimization. Cost is dominated by *how much* rendering the pipeline requests per frame. Since SLAM does not require photorealism, the render budget itself becomes the optimization opportunity.

1.1 Our Approach:

SLAM observes a scene incrementally. Each frame captures a partial, overlapping view, and the map is built by integrating structure across many such observations over time. Consequently, no single frame must reconstruct the full depth complexity or spatial resolution of the scene: the dominant structure—geometry, large-scale color, and depth discontinuities—can be captured at reduced fidelity, while high-frequency texture and distant occluded structure are observed again from subsequent viewpoints. This is fundamentally different from offline novel-view synthesis, where every rendered image is a final output and must be photorealistic. This observation motivates render budgeting as a SLAM-aware acceleration abstraction: allocate fewer pixels and fewer compositing contributions per frame, and let temporal integration recover what any single frame omits.

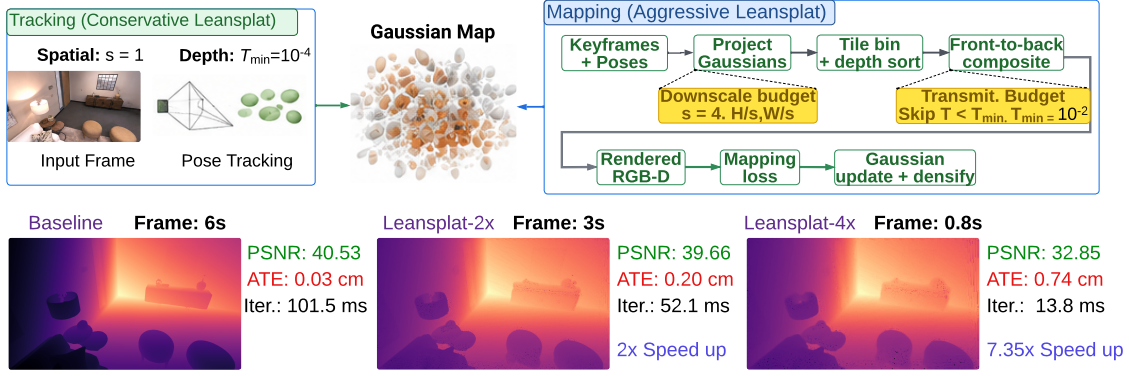


Figure 1: Overview of LeanSplat. Tracking uses conservative setting, while mapping uses reduced spatial and depth budgets.

Figure 1 illustrates this idea. LeanSplat allocates different rendering budgets to tracking and mapping, reducing rendering workload through spatial downscaling and early transmittance termination while preserving accurate pose estimation. We realize this abstraction in *LeanSplat* through two complementary mechanisms. *Image downscaling* renders at reduced spatial resolution ($H/s \times W/s$), cutting pixel count by approximately s^2 and eliminating redundant spatial information that neighboring pixels share. *Early transmittance termination* stops compositing a pixel once its accumulated transmittance falls below a threshold $T_{\min}$, skipping distant or heavily occluded gaussians whose bounded contributions will be observed from future frames. Together these control both the spatial budget (fewer pixels) and the depth budget (fewer contributors per pixel) of each rendering pass. Unlike reduction-tree compositing, which reorganizes the same computation, LeanSplat changes *how much* computation is performed. Unlike representation-compression methods, it leaves the underlying gaussian scene model unchanged; unlike alternative compositing schemes, it preserves the standard ordered front-to-back accumulation and simply budgets when rendering stops. This makes LeanSplat better matched to incremental SLAM, where budgets can be adjusted online, tuned differently for tracking and mapping, and applied while the scene is still being optimized.

Uniform render budgets expose an asymmetry between tracking and mapping: tracking compounds rendering errors into pose drift, while mapping integrates structure across frames and tolerates lower per-frame fidelity. On Replica room0, uniform reduction destabilizes SplatAM, but full-resolution tracking with reduced-budget mapping keeps ATE to 0.20cm at $2\times$ and 0.74cm at $4\times$ over 50 frames, while cutting mapping time from 6.09s to 3.13s and 0.83s. This motivates an *asymmetric* budget: conservative tracking and aggressive mapping. In standalone gaussian splatting across two datasets [16, 17], LeanSplat raises throughput from 18–24 to 50–67 iterations/s ($2.08\times$ – $3.22\times$) with only 0.06–0.54 dB PSNR loss. Our contributions:

- We propose LeanSplat, a SLAM-oriented render-budgeting method using image downscaling and early transmittance termination. Without modifying the gaussian representation or compositing order, it achieves $2.08\times$ – $3.22\times$ speedups with controlled fidelity loss.
- We present evidence from SplatAM integration that *LeanSplat* can maintain trajectory accuracy (0.74cm) while achieving up to $7.35\times$ lower mapping-frame time. We also find evidence that tracking is more sensitive to render budgeting than mapping, motivating an asymmetric render budget.

2 Motivation and Scope

Takeaway: Although gaussian splatting replaces volumetric ray marching with efficient rasterization, it still composites long stacks of depth-sorted contributors per pixel—work that is repeated during both tracking and mapping. Yet most opacity accumulates early in each stack, motivating a budgeted approach that reduces per-frame workload rather than reorganizing the same computation.

Figure 2 measures the effective compositing depth on Replica room0: for each pixel, we count the gaussian contributors processed before accumulated transmittance falls below 0.0001. Many regions require deep stacks rather than a few dominant contributors. The median pixel processes about 77 contributors and the

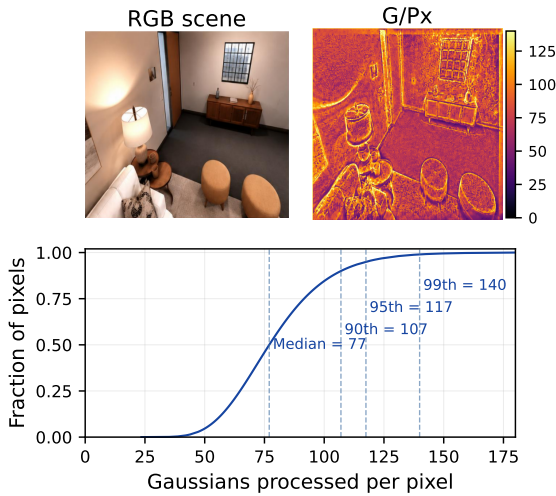


Figure 2: Distribution of gaussian contributors per pixel on Replica room0.

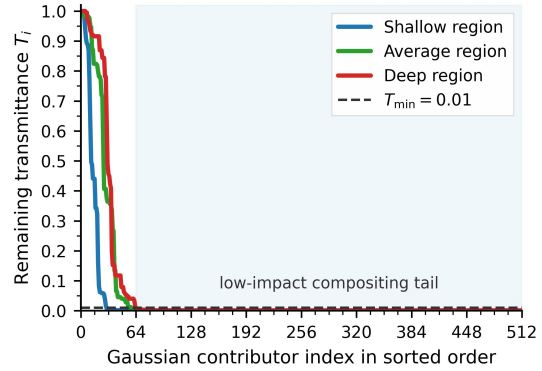


Figure 3: Remaining transmittance versus gaussian contributor index for representative scene regions.

90th percentile reaches 107 before transmittance becomes negligible. The bottleneck is therefore the amount of work requested per frame, not the arithmetic cost of alpha blending. Even though distant contributors barely affect the final color, the renderer cannot skip them because transmittance accumulates sequentially: $T_i = \prod_{j < i} (1 - \alpha_j)$.

Figure 3 suggests an additional opportunity. Most pixels accumulate the bulk of their opacity early in the sequence, leaving a long low-impact tail. Because SLAM integrates information incrementally across overlapping frames, reconstructing the full compositing depth every frame may be unnecessary. This motivates LeanSplat: rather than reorganizing the same computation, we reduce the rendering workload along two axes: (i) fewer rendered pixels via image downscaling, and (ii) fewer processed contributors via early transmittance termination. The key idea is to trade per-frame fidelity for reduced compositing cost while preserving the geometric and photometric structure SLAM needs.

2.1 Related Work

SLAM has evolved from sparse feature-based pipelines to dense, neural, and gaussian-based mapping. Early systems [3, 4] tracked efficiently from sparse features but produced maps with limited geometric detail. Dense SLAM recovered richer geometry through explicit map structures [18, 19, 5, 6] at higher computational cost. Neural representations later enabled high-quality mapping [7, 8], but ray sampling makes real-time infeasible.

Gaussian splatting represents scenes with anisotropic gaussian primitives that rasterize efficiently [12], making it attractive for real-time mapping and SLAM [13, 14, 15]. Rendering nevertheless requires compositing many depth-sorted contributions per pixel, which becomes the bottleneck. Recent works speedup splatting in two ways: (i) shrinking the representation through pruning [20, 21, 22], which targets offline or memory-constrained rendering; and (ii) speeding up rasterization by simplifying sorting, approximating visibility, or reducing alpha-blending cost.

To our knowledge, *LeanSplat* is the first to frame SLAM acceleration as online render budgeting—reducing the work *requested* per frame rather than compressing the representation or reorganizing the compositing arithmetic. It applies this budgeting along two axes: fewer rendered pixels via image downscaling, and fewer processed contributors via early transmittance termination. This aligns naturally with incremental SLAM: budgets adjust at runtime, can differ between tracking and mapping, and require no recompression of a scene that is still being optimized. *LeanSplat* remains complementary to compact gaussian representations and offline acceleration methods. *LeanSplat* is orthogonal to the choice of gaussian SLAM backbone: it modifies neither the representation nor the compositing order, only the per-call render budget. We therefore evaluate it inside SplatAM [13] as a drop-in modification rather than as a competing system, and expect the same budgets to transfer to backbones such as MonoGS [14] and Photo-SLAM [15] that also separate tracking and mapping render passes.

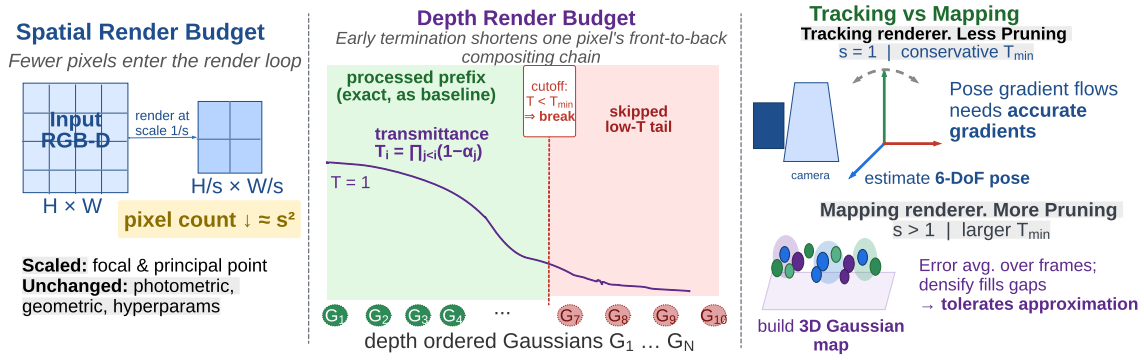


Figure 4: LeanSplat render budgeting. Spatial downscaling reduces rendered pixels, while early termination reduces processed gaussian contributors. Asymmetric budgets between mapping and tracking.

3 LeanSplat: Budgeted Gaussian Splatting for SLAM

LeanSplat is a render-budgeting method for gaussian-based SLAM. It reduces work in the standard gaussian renderer along two axes: spatial downscaling cuts the number of pixels rendered, and early transmittance termination cuts the number of gaussian contributors processed per pixel. Figure 4 summarizes the strategy.

3.1 Spatial Render Budget

In 3D gaussian splatting [12], a pixel p is rendered by compositing depth-sorted gaussian contributors in front-to-back order. The accumulated color is $C(p) = \sum_{i=1}^N T_i \alpha_i c_i$, where c_i is the color of the i -th gaussian, α_i is its projected opacity at p , and $T_i = \prod_{j<i} (1 - \alpha_j)$ is the accumulated transmittance before that contributor. Figure 2 shows that many pixels require long compositing chains before transmittance becomes negligible.

Given an input image of size (H, W) , LeanSplat renders at reduced spatial resolution $H' = H/s$, $W' = W/s$, where s is the downscale factor, reducing the number of pixels by approximately s^2 . Camera intrinsics are scaled accordingly (focal lengths and principal-point coordinates divided by s), and the RGB-D target is downsampled to $H' \times W'$ before computing photometric and geometric losses. Losses, optimizer, densification schedule, and training hyperparameters are unchanged—spatial budgeting is an isolated rendering-side intervention.

For mapping, neighboring pixels often repeat structural information. Depth discontinuities, surface orientation, and large-scale color usually vary smoothly over local image regions. High-frequency texture changes faster, but matters less for coarse geometric reconstruction. Reduced-resolution rendering can therefore keep the structure needed for mapping while cutting repeated per-pixel work. Figure 6 shows that reduced-budget rendering mainly removes high-frequency details while preserving scene structure.

Spatial downscaling provides most of the observed speedup because pixel count falls quadratically with s , while early termination adds a scene-dependent benefit where pixels have long compositing tails. Since the two mechanisms act on different axes, their speedups compose approximately multiplicatively when both expose reducible work.

3.2 Depth Render Budget

LeanSplat also reduces contributors per pixel. During front-to-back compositing, transmittance is updated as $T \leftarrow T(1 - \alpha_i)$. Once the remaining transmittance falls below a threshold $T_{\min}$, compositing terminates: $T < T_{\min}$. The threshold $T_{\min}$ bounds the worst-case per-pixel approximation error. Once transmittance reaches $T_{\min}$, all unprocessed gaussians contribute at most $T_{\min}$ per color channel (assuming RGB values in $[0, 1]$), so $T_{\min} = 0.01$ caps the remaining color contribution at about 1% of the dynamic range.

Early termination differs structurally from contribution-based pruning. Pruning removes individual contributors from arbitrary positions, creating gaps that alter the transmittance seen by later contributors and can destabilize optimization. Early termination instead preserves a contiguous front-to-back prefix: contributors up to the cutoff are processed exactly as in the baseline, and only the low-transmittance tail is skipped, so gradients pass through the same ordered chain over a shorter depth range.

Figure 5: LeanSplat Renderer.

Algorithm 5 gives the complete renderer; it matches the standard 3DGS compositing loop except for the reduced-resolution target (line 1) and the transmittance check that halts compositing when $T_p < T_{\min}$ (line 12). For each reduced-resolution pixel, the renderer composites the depth-sorted overlapping gaussians front to back, skipping negligible opacities ($\alpha_i < 1/255$, inherited from baseline 3DGS) and halting once incoming transmittance falls below $T_{\min}$. The baseline runs until the gaussian list is exhausted or a near-zero default threshold is reached; LeanSplat raises $T_{\min}$ and usually stops earlier. For a 90th-percentile pixel that processes 107 contributors in the baseline (Fig. 2), a cutoff after 30–50 contributors removes roughly 50–70 low-impact terms. The termination check precedes the transmittance update, so every processed contributor has incoming transmittance at least $T_{\min}$. Line 15 uses SplaTAM’s median-depth heuristic: depth is set to z_i when accumulated compositing weight first exceeds 0.5. This depth is used for tracking in Section 4.1; standalone 3DGS experiments evaluate color reconstruction.

The opacity check and early termination are complementary: the opacity check removes isolated negligible gaussians anywhere in the sequence, while early termination halts the loop once the pixel is effectively opaque. Both are implemented, but depth-workload savings mainly arise in scenes with deep compositing tails. The two axes remain approximately independent in practice. Although a scale- s pixel covers an $s \times s$ original-image region, per-pixel compositing depth did not substantially change in our scenes, so we use the same $T_{\min}$ across downscale factors. Table 3 shows that the combined $2\times$, $T_{\min} = 0.01$ setting reaches a $2.25\times$ speedup on Truck, consistent with separate workload axes. Setting $s = 1$ disables downscaling. Setting $T_{\min} = 0.0001$ recovers the baseline 3DGS near-zero termination threshold. With both settings, Algorithm 5 reduces to the standard front-to-back loop. LeanSplat is therefore a two-parameter extension of the existing renderer, not a new rendering algorithm.

3.3 Tracking and Mapping Implications

LeanSplat should be applied asymmetrically—conservatively for tracking, aggressively for mapping—because the two SLAM stages use rendered images differently. Tracking estimates the six camera-pose parameters by minimizing photometric and geometric losses between the render and the current observation. The pose gradient flows through the rendered image, so distortions from downscaling or shallow compositing yield noisy updates that miss the true pose; errors compound as a bad pose shifts the next rendered viewpoint. Section 4.1 validates this asymmetry: globally reduced rendering produces ATE RMSE in the hundreds of centimeters, while full-resolution tracking with $2\times$ mapping holds ATE to 0.20 cm on the same 50-frame trajectory (Table 1).

Mapping optimizes gaussian position, covariance, opacity, and color across many frames and viewpoints, making it more tolerant to reduced per-frame fidelity. Single-frame errors are diluted by the multi-frame average, and SplaTAM densification can add gaussians to underrepresented regions, partially compensating for structures missed in individual shallow renders. Mapping can therefore absorb moderate approximation, provided the poses are accurate. Implementation is straightforward: SplaTAM [13] already calls the renderer separately for tracking and mapping with independent parameters, so asymmetric rendering only requires different downscale factors and $T_{\min}$ values for those calls. The same strategy applies to other gaussian SLAM systems with separate tracking and mapping rendering passes. We evaluate full-resolution tracking as the conservative endpoint and reduced-budget mapping as the aggressive endpoint. Mild tracking reduction (e.g., $1.5\times$ downscaling or a modestly raised $T_{\min}$) requires a finer parameter sweep and is left to future work. Section 4.1 shows that the conservative asymmetric design preserves pose stability while lowering cost.

Require: Camera parameters $\mathcal{P}$, Gaussian set $\mathcal{G}$, downscale factor $s \geq 1$, threshold $T_{\min} \in (0, 1)$
Ensure: Rendered color image C and depth map D
1: $H' \leftarrow \lfloor H/s \rfloor$, $W' \leftarrow \lfloor W/s \rfloor$ ▷ Downscaling
2: Initialize $C \leftarrow 0$, $D \leftarrow +\infty$, and $T \leftarrow 1$ over $H' \times W'$ ▷ Initialize buffers
3: Project all Gaussians ($\mathcal{G}$) using $\mathcal{P}$ ▷ Visibility and projection
4: Bin Gaussians into tiles and depth-sort ▷ Per-tile sorting
5: **for** each pixel $p = (x, y)$ in the reduced-resolution image **do** ▷ Pixel loop
6: Retrieve ordered list $\mathcal{G}_p = [G_1, G_2, \dots, G_{N_p}]$ ▷ Overlapping Gaussians
7: $C_p \leftarrow (0, 0, 0)$, $D_p \leftarrow +\infty$, $T_p \leftarrow 1$ ▷ Per-pixel init
8: **for** each Gaussian $G_i \in \mathcal{G}_p$ **do** ▷ Compositing loop
9: Evaluate G_i at p and compute ▷ Conic and opacity
10: $\alpha_i \leftarrow o_i \exp(-\frac{1}{2} \mathbf{u}_i^T \Sigma_i^{-1} \mathbf{u}_i)$ ▷ $\alpha_i \in [0, 1]$
11: **if** $\alpha_i < 1/255$ **then continue** ▷ Skip negligible contribution
12: **if** $T_p < T_{\min}$ **then break** ▷ Early termination
13: $w_i \leftarrow T_p \alpha_i$ ▷ Contribution weight
14: $C_p \leftarrow C_p + w_i G_i$ ▷ Color accumulation
15: **if** $D_p = +\infty$ **and** $1 - T_p + w_i \geq 0.5$ **then** $D_p \leftarrow z_i$ ▷ Median depth
16: $T_p \leftarrow T_p(1 - \alpha_i)$ ▷ Transmittance update
17: $C(x, y) \leftarrow C_p$, $D(x, y) \leftarrow D_p$ ▷ Write pixel

4 Experiments and Evaluation

We conduct all experiments on an NVIDIA Jetson AGX Thor <https://developer.nvidia.com/embedded/jetson-developer-kits> with CUDA 13.0 and PyTorch 2.9.1. SplaTAM experiments use the github commit (da6bbcd) [13]. We use the image sequences at the native resolution of 680×1200 . We evaluate on truncated Replica trajectories (10, 50, and 100 frames) rather than full sequences. Short trajectories isolate the effect of per-frame render budgeting from long-horizon dynamics — loop closure, drift accumulation, and global refinement — that are orthogonal to our contribution, and they make a dense budget sweep tractable. Because of this, our ATE values are not directly comparable to full-sequence SplaTAM benchmarks; we therefore emphasize relative speedup and within-protocol accuracy rather than absolute leaderboard ATE. For inputs, we use scenes from Tanks&Temples [16], and Blending dataset [17]. For SplaTAM we report Absolute Trajectory Error (ATE), Depth error (D-RMSE) and per-frame mapping time. In addition, we measure gaussian splatting efficacy on 3DGS [12]; we report iteration speed (it/s), training loss, PSNR (signal-to-noise ratio), MS-SSIM (Multi-Scale Structural Similarity Index Measure), and LPIPS (Learned Perceptual Image Patch Similarity).

4.1 LeanSplat Efficacy in SplaTAM

Takeaway 1: *LeanSplat maintains sub-centimeter trajectory accuracy (0.74cm) at $4\times$ while cutting mapping time by up to $7.35\times$.*

Because SplaTAM uses rendered RGB-D for both tracking and mapping, rendering approximations can globally degrade tracking: small pose errors from downsampled or truncated maps compound as inaccurate poses further degrade the map. We therefore also evaluate *LeanSplat* with ground-truth poses, isolating mapping quality from tracking sensitivity. Under fixed poses, *LeanSplat* preserves coarse geometry and remains usable on short sequences, while longer horizons expose a clearer quality–speed tradeoff. LPIPS values are not directly comparable across scales, since downscaled settings are evaluated against downsampled targets.

Table 1(a) reports the asymmetric configuration without GT poses: full-resolution tracking with $2\times$ or $4\times$ reduced mapping. Relative to the full-resolution 50-frame baseline, the $2\times$ setting raises ATE only from 0.03cm to 0.20cm (comparable PSNR and depth accuracy). The more aggressive $4\times$ setting still holds sub-centimeter accuracy (0.74cm ATE) while cutting mapping-frame time from 6.09s to 0.83s — a $7.35\times$ speedup, at the cost of a moderate PSNR reduction (40.53→32.85).

Table 1(b) shows why global reduction fails: even when PSNR stays reasonable, ATE explodes to orders of magnitude above the baseline. The rows in (b) are failure-mode illustrations rather than matched comparisons — frame counts differ (100/100/50/10) because each setting fails at a different horizon, and the ordering be-

Table 1: SplaTAM w/ *LeanSplat*.

(a) Asymmetric Render Budgeting									(b) Global Render Budgeting					
Budget	$T_{\min}$	Frames	ATE	Time	Speedup	PSNR	D-RMSE	LPIPS	Budget	Frames	ATE	PSNR	RMSE	LPIPS
Full, Full	0.0001	50	0.03cm	6.09s	1.00×	40.53	0.17cm	0.021	$4\times$	100	161.14cm	13.80	60.27cm	0.681
Full, $2\times\downarrow$	0.01	50	0.20cm	3.13s	1.95×	39.66	0.32cm	0.008	$2\times$	100	397.25cm	13.88	76.88cm	0.612
Full, $4\times\downarrow$	0.01	50	0.74cm	0.83s	7.35×	32.85	0.67cm	0.018	$2\times$	50	753.88cm	27.13	19.99cm	0.238
									Full	10	1.45cm	19.81	13.43cm	0.355

(c) Cross-Scene Validation							(d) Mapping Under GT Poses					
Scene	ATE _B	ATE _{2×}	ATE _{4×}	Time _B	Time _{2×}	Time _{4×}	Setting	Map(s)	PSNR	D-RMSE	MS-SSIM	LPIPS
							10 frames					
room0	0.03cm	0.20cm	0.74cm	6.09s	3.13s	0.83s	Full, $T_{\min} = 0.001$, GT pose	6.34s	38.61	3.96cm	0.995	0.047
room1	0.12cm	0.24cm	0.30cm	7.66s	1.94s	0.81s	Full, $T_{\min} = 0.01$, GT pose	6.09s	27.95	10.30cm	0.965	0.207
room2	0.08cm	0.13cm	0.40cm	7.20s	1.89s	0.83s	$2\times$, $T_{\min} = 0.01$, GT pose	1.69s	28.54	11.63cm	0.979	0.152
office0	0.10cm	0.42cm	1.00cm	7.43s	1.89s	0.82s	$4\times$, $T_{\min} = 0.01$, GT pose	0.80s	26.54	14.39cm	0.982	0.144
office1	0.08cm	0.22cm	0.55cm	7.53s	1.95s	0.82s						
							50 frames					
Mean	0.08cm	0.24cm	0.60cm	7.18s	2.16s	0.82s	Full, $T_{\min} = 0.001$, Control	10.62s	40.68	0.17cm	0.996	0.021
							$2\times$, $T_{\min} = 0.01$, GT pose	3.12s	22.02	44.24cm	0.803	0.457
							$4\times$, $T_{\min} = 0.01$, GT pose	1.02s	21.59	40.16cm	0.822	0.454

tween rows is not meaningful; the point is that every global-budgeting setting fails catastrophically. Together these results establish the central system-level insight: tracking and mapping have fundamentally different tolerance to rendering approximation, so *LeanSplat* should target mapping rather than pose estimation. To test whether this trend is specific to `room0`, we repeat the same 50-frame asymmetric mapping experiment across five Replica scenes. Table 1(c) shows that the same accuracy–speed tradeoff holds across other datasets.

Table 1(d) reports mapping under fixed ground-truth poses, optimizing only the gaussians; since Table 1(a) co-optimizes estimated poses, absolute fidelity is not comparable across the two. With reliable poses, *LeanSplat* yields large speedups: at 10 frames, $2\times$ reaches 1.69 s per mapping frame (PSNR 28.54) and $4\times$ reaches 0.80 s (PSNR 26.54). The depth budget dominates fidelity: at full resolution, raising $T_{\min}$ from 0.001 to 0.01 drops PSNR by over 10 dB (38.61 $\rightarrow$ 27.95). The spatial budget has a far smaller fidelity effect — at fixed $T_{\min}=0.01$, $2\times$ downscaling changes PSNR by under 1 dB — though we caution that down-scaled PSNR is measured against downsampled targets and is not directly comparable across scales (see above). Over 50 frames both budgeted settings degrade, reflecting the aggressive depth budget compounding with long-horizon accumulation rather than *LeanSplat* alone; the 50-frame control ($T_{\min} = 0.001$, PSNR 40.68) stays healthy under a gentle budget. They map the boundary cases, not recommended operating point.

4.2 Comparing Gaussian Splatting Efficacy

We compare *LeanSplat* against two alternatives: reduction-tree compositing and contribution-based pruning. i) **Reduction-Tree Compositing:** Because transmittance is a prefix product, $T_i = \prod_{j<i}(1 - \alpha_j)$, compositing can be expressed as a parallel prefix operation. We implement this directly in CUDA with warp-level scans while keeping the same rendering workload: the same pixels and depth-sorted lists. In practice, synchronization, shuffles, and bookkeeping largely erase the theoretical gain; only marginal improvement. ii) **Contribution-Based Pruning:** This approach skips terms with small effective contribution, $\alpha_i T_i < \tau$. However, it does not bound per-pixel work because the renderer must still traverse the list to test them. It also removes contributors at arbitrary depths, changing later transmittance and reducing optimization stability. *LeanSplat* preserves the gaussian ordering front-to-back and drops only the low-impact tail.

Takeaway 2: *LeanSplat delivers $2.08\times$ – $3.22\times$ speedups across four scenes with small PSNR loss, while reduction-tree compositing and contribution-based pruning are ineffective.*

Takeaway 3: *The speedup generalizes across all four scenes ($2.08\times$ – $3.22\times$) and is largest on Drjohnson, where higher depth complexity creates longer compositing tails for *LeanSplat* to remove. *LeanSplat* adapts to scene-dependent depth complexity and rendering workload.*

Table 2 summarizes the main results. The baseline converges stably at 18–24 it/s across the four scenes (Table 3). Reduction-tree reaches only 25 it/s versus 24 on Truck, confirming that reorganizing the same computation does not accelerate it. Pruning shows similarly marginal speedups but higher loss variance, indicating less stable optimization. Table 3 extends the evaluation to Train, Drjohnson, and Playroom. Every scene speeds up with small PSNR loss, but the magnitude tracks depth complexity. Drjohnson gains the

Table 2: Gaussian Splatting comparison.

Method	Scale	$T_{\min}$	Iter/s	Loss	PSNR	MS-SSIM	LPIPS
Baseline	$1\times$	0.0001	24	0.0298	25.31	0.8581	0.1516
Reduction Tree	$1\times$	0.0001	25	0.0297	25.30	0.8583	0.1519
αT Pruning	$1\times$	0.0001	25	0.0301	25.26	0.8584	0.1525
<i>LeanSplat</i>							
Ours	$2\times$	0.01	54	0.0317	24.97	0.86	0.0940
Ours	$4\times$	0.01	67	0.0383	23.37	0.85	0.0947

Table 3: Cross-scene evaluation of *LeanSplat*.

Scene	Baseline Iter/s	Ours Iter/s	PSNR Drop	Speedup
Truck	24	54	0.3359	2.25x
Train	24	50	0.2800	2.08x
Drjohnson	18	58	0.0563	3.22x
Playroom	21	67	0.5425	3.19x



Figure 6: Qualitative comparison of *LeanSplat* on the Truck scene.

(a) Downscaling				
Scale	It/s	Loss	PSNR	LPIPS
2×	54	0.0317	24.9519	0.0940
4×	67	0.0383	23.3681	0.0939

(b) $T_{\min}$ early termination				
$T_{\min}$	It/s	Loss	PSNR	LPIPS
0.001	25.1	0.0300	25.2736	0.1521
0.01	25.2	0.0301	25.2564	0.1512
0.02	25.3	0.0297	25.3217	0.1512

Table 4: Downscaling vs. Early-termination.

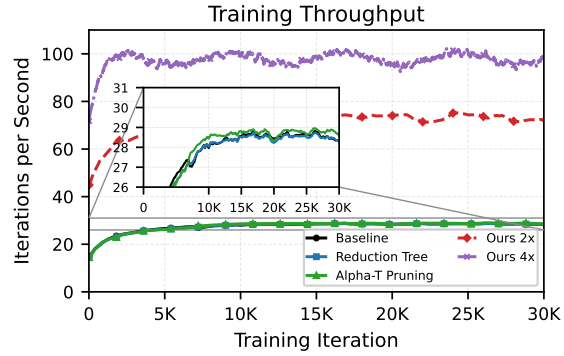


Figure 7: Training throughput during optimization.

most ($3.22\times$ at 0.06 dB), as heavier per-pixel compositing offers more low-impact contributions to remove; Playroom hits the highest throughput (67 it/s) but the largest drop (0.54 dB) from stripping high-frequency structure; Truck and Train sit between at roughly $2\times$. All four converge stably, showing *LeanSplat* generalizes across spatial structure and depth complexity. Figure 6 confirms that the dominant geometric structure survives *LeanSplat* despite the loss of fine-scale depth detail.

4.3 Ablation Studies

Downscaling reduces pixel workload and improves speed while preserving PSNR (Table 4(a)), but the gains are sub-linear: $2\times$ downscaling removes $4\times$ the pixels yet improves throughput only from 24 to 54 it/s ($2.25\times$), while $4\times$ removes $16\times$ the pixels for 67 it/s ($2.79\times$). This points to pixel-independent costs from sorting, tile binning, synchronization, and setup. The limited quality loss suggests that neighboring pixels are redundant enough to preserve SLAM-relevant geometry and appearance, despite reduced detail.

Early termination helps mainly in scenes with long compositing tails. Early termination shrinks the effective compositing range: larger $T_{\min}$ stops earlier and removes deeper low-impact contributions. Table 4(b) shows little gain on Truck, consistent with moderate depth complexity and short tails; Drjohnson shows the opposite behavior in Sec. 4.2. Overly aggressive thresholds still hurt convergence and quality, so early termination is range reduction rather than arbitrary pruning, matching Figure 3. To study stability, we compare training throughput in Figure 7. We find that preserving that ordered structure matters more for stable optimization than aggressively removing individual contributions.

5 Limitations and Future Directions

LeanSplat separates the broader idea of *asymmetric render budgeting* from the specific mechanisms used here. Tracking and mapping tolerate approximation differently, so they should receive decoupled budgets; we instantiate this with spatial downscaling and early transmittance termination. These mechanisms already reach ≈ 70 it/s and up to a $7.35\times$ mapping speedup, but pushing further has limits. Aggressive downscaling removes high-frequency texture and fine geometry, although the dominant structure needed for SLAM is preserved (Figure 6). Early termination can discard distant or partially occluded structure, and applying either approximation globally can destabilize pose estimation over long trajectories. These failures are most likely in scenes with thin geometry, detailed textures, reflections, or ambiguous alignment cues. They reflect the fixed policies evaluated here rather than the budgeting principle itself. Future systems could allocate render budgets adaptively based on scene complexity, motion uncertainty, visibility, or SLAM stage. Our evaluation uses short trajectories; longer-term SLAM may require more careful budgeting to prevent divergence.

6 Conclusion

This work argues that accelerating gaussian splatting for SLAM is primarily a render-budgeting problem: because SLAM observes scenes incrementally, each frame need not render the full spatial and depth complexity of the scene. *LeanSplat* exploits this with image downscaling and early transmittance termination to trade per-frame photorealism for speed.

References

- [1] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016.
- [2] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse. Monoslam: Real-time single camera slam. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1052–1067, 2007.
- [3] G. Klein and D. Murray. Parallel tracking and mapping for small ar workspaces. In *IEEE/ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 225–234, 2007.
- [4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos. Orb-slam: A versatile and accurate monocular slam system. *IEEE Transactions on Robotics*, 31(5):1147–1163, 2015.
- [5] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison. Dtam: Dense tracking and mapping in real-time. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2320–2327, 2011.
- [6] J. Engel, T. Schöps, and D. Cremers. Lsd-slam: Large-scale direct monocular slam. In *European Conference on Computer Vision (ECCV)*, pages 834–849, 2014.
- [7] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421, 2020.
- [8] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5470–5479, 2022.
- [9] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison. imap: Implicit mapping and positioning in real-time. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6229–6238, 2021.
- [10] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys. Nice-slam: Neural implicit scalable encoding for slam. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12786–12796, 2022.
- [11] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald. Point-slam: Dense neural point cloud-based slam. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 18433–18444, 2023.
- [12] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 42(4), 2023.
- [13] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten. Splatam: Splat, track & map 3d gaussians for dense rgb-d slam. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [14] H. Matsuki, R. Murai, J. Kelly, and A. J. Davison. Gaussian splatting slam (monogs). In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [15] H. Huang, L. Li, H. Cheng, and S.-K. Yeung. Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [16] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. In *ACM Transactions on Graphics (SIGGRAPH)*, 2017.
- [17] G. Hedman, J. Philip, T. Price, J.-M. Frahm, G. Drettakis, and G. Brostow. Deep blending for free-viewpoint image-based rendering. In *ACM Transactions on Graphics (SIGGRAPH)*, 2018.

- 340 [18] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton,
341 S. Hodges, and A. W. Fitzgibbon. Kinectfusion: Real-time dense surface mapping and tracking. In
342 *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 127–136, 2011.
- 343 [19] T. Whelan, S. Leutenegger, R. Salas-Moreno, B. Glocker, and A. Davison. Elasticfusion: Dense slam
344 without a pose graph. In *Robotics: Science and Systems (RSS)*, 2015.
- 345 [20] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang. LightGaussian: Unbounded 3D gaussian
346 compression with 15x reduction and 200+ FPS. In *Advances in Neural Information Processing Systems*
347 *(NeurIPS)*, 2024.
- 348 [21] G. Fang and B. Wang. Mini-Splatting: Representing scenes with a constrained number of gaussians. In
349 *European Conference on Computer Vision (ECCV)*, 2024.
- 350 [22] K. L. Navaneet, K. Pourahmadi Meibodi, S. A. Koochpayegani, and H. Pirsiavash. CompGS: Smaller
351 and faster gaussian splatting with vector quantization. In *European Conference on Computer Vision*
352 *(ECCV)*, 2024.